perplexity

Step 4: Types and strategies

Regression testing is not one fixed method. Teams choose a **scope**—from a few targeted checks to the whole application—based on how risky and widespread the change is. [1] [2]

Main types

Type	What it means	When to use it	Example
Corrective regression	Reruns existing tests without changing them	The code changed, but requirements and expected behavior did not	A developer refactors code internally; users should see no difference
Selective regression	Runs only tests related to the changed code and its dependencies	A localized bug fix or small feature change	A change to the search API triggers search and results-page tests
Progressive regression	Adds tests for new functionality while checking old features it touches	A new feature is introduced	A new "Save for later" cart option is tested alongside normal add-to-cart and checkout
Complete/full regression	Runs the broad regression suite across the application	Major releases, upgrades, migrations, or high-risk changes	Before launching a new app version or after a database migration
Smoke regression	Runs a small set of critical checks	After every build or deployment	Verify login, home page, product search, and checkout load correctly
Unit regression	Tests individual functions or classes	After changes to a specific unit of code	A pricing function is tested after its calculation logic changes [1] [2] [3]

Retesting vs regression testing

These terms are often confused, but they answer different questions. **Retesting** verifies that one specific bug is fixed; **regression testing** verifies that the change did not break other previously working behavior. [4] [5]

Aspect	Retesting	Regression testing
Main question	"Is this bug gone?"	"Did this change break anything else?"
Trigger	A defect is fixed	Any code, configuration, or dependency change
Scope	The failed test or defect scenario	Changed area, dependent areas, and critical flows

Aspect	Retesting	Regression testing
Typical order	Usually first	Usually after retesting confirms the fix
Example	Check that a wrong cart total now displays correctly	Check discounts, taxes, shipping, payment, receipts, and order history after the cart-total fix [4] [6] [5]

Example: if a discount calculation bug is fixed, **retesting** confirms the original discount case now works. **Regression testing** then checks other discounts, taxes, shipping, checkout totals, emails, and refunds—because the shared calculation code could affect all of them.[6] [7]

Choosing a strategy

A good beginner strategy is **risk-based regression testing**: spend the most testing time where failure would cause the most harm. Teams map core user journeys first, then prioritize tests by business impact, likelihood of failure, and how often the area changes.[2] [8]

Use this simple decision guide:

Change size	Recommended approach
Very small UI text or styling change	Targeted visual/UI check plus smoke tests
Small bug fix	Retest the bug, then selective regression for related flows
New feature	Progressive regression: new tests plus tests for connected old features
Dependency, framework, or database upgrade	Broad or complete regression suite
Major release	Smoke tests first, then full regression suite [1] [2] [3]

Practical layering

Many teams use layers rather than choosing only one type:

- **Every commit or pull request:** fast unit and smoke tests.
- **Merge to main branch:** targeted and selective regression tests.
- **Nightly or scheduled run:** broader regression suite.
- **Before release:** complete regression suite and critical end-to-end journeys.[7] [2]

This balances speed with confidence: developers get quick feedback, while the team still catches broader side effects before release.[2] [7]

Step 5 preview

Next, I can explain **Step 5: Tools, automation, and best practices**, including:

- Popular tools such as Selenium, Playwright, Cypress, JUnit, Pytest, and CI/CD systems
- How automated regression tests fit into GitHub Actions, Jenkins, or GitLab CI
- How to avoid flaky tests

- Best practices for maintaining a regression suite

Would you like to continue to **Step 5**?

⁂

1. https://www.inflectra.com/Ideas/Topic/What-is-Regression-Testing.aspx
2. https://www.virtuosoqa.com/post/types-of-regression-testing
3. https://keploy.io/blog/community/types-of-regression-testing-in-software-testing
4. https://quashbugs.com/blog/retesting-vs-regression-testing
5. http://www.browserstack.com/guide/retesting-vs-regression-testing
6. https://www.virtuosoqa.com/post/regression-testing-vs-retesting
7. https://prepare.sh/articles/what-is-regression-testing
8. https://www.flagsmith.com/blog/automated-regression-testing
9. https://www.drizz.dev/post/retesting-vs-regression-testing
10. http://eprints.intimal.edu.my/2284/1/joit2025_34.pdf
11. https://www.opkey.com/blog/what-is-regression-testing-why-should-it-be-automated
12. https://katalon.com/resources-center/blog/what-is-retesting
13. https://www.ibm.com/think/topics/regression-testing
14. https://testomat.io/blog/regression-testing-vs-retesting/
15. https://www.virtuosoqa.com/post/basics-of-regression-testing
16. https://momentic.ai/blog/types-of-regression-testing